



SIMATS ENGINEERING

Short Answer Test

COURSE NAME: Software Testing

COURSE CODE: CSA3711

COURSE OUTCOME COVERED: CO1: Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.

Assessment Tool Weightage: 30%

Code of Conduct

I Rakcha A (192311421) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Rakcha A

Q1. A startup team relies on random testing methods, leading to inconsistent results and missed defects. The project manager wants to improve testing effectiveness. As a test engineer, apply the concept of testing as an engineering activity to redesign the approach. Explain how structured test design improves quality and defect detection.

Answer:

Testing is not an ad-hoc activity of “trying things out”; it is an engineering activity that is planned, designed, executed, measured and improved, just like design or coding. The random approach of the startup is therefore replaced with the following engineered approach:

- **Analyse requirements:** identify testable features, risks and acceptance criteria.
- **Plan:** define scope, objectives, resources, schedule, entry/exit criteria and the test environment.
- **Design test cases systematically:** use techniques such as equivalence partitioning, boundary value analysis, decision tables and state-transition testing, with clear inputs, preconditions and expected results.
- **Execute and record:** run the tests, log actual results and report defects in a tracking tool.
- **Measure and improve:** track coverage, defect density and pass/fail trends, and refine the tests in the next cycle.

How structured test design improves quality: Every requirement is traced to at least one test case, so nothing is missed. Test cases are repeatable, so results are consistent between testers and releases. Techniques generate a small but powerful set of test cases that target the places where defects typically hide (boundaries, invalid classes, rule combinations). Coverage can be measured, so the team knows when testing is sufficient. For example, a signup form with an age field of 18–60 is tested with 17, 18, 19, 59, 60 and 61 instead of random values; this finds boundary defects that random testing would probably miss.

Q2. A banking system undergoes multiple failures due to lack of a defined testing process. The organization plans to standardize its testing activities. Illustrate how a well-defined testing

process improves software quality in such real-time systems. Explain the benefits with relevant examples.

Answer:

A defined testing process is a set of ordered, documented activities that every project follows: test planning and control, test analysis and design, test environment and data setup, test execution, defect reporting and tracking, and test closure with lessons learned.

Illustration in a banking system:

- **Planning:** critical modules such as fund transfer, interest calculation, loan processing and ATM/UPI interfaces are identified as high risk and get more test effort.
- **Design:** test cases cover normal flows, boundary limits (minimum balance, daily transfer limit), invalid inputs and failure scenarios such as a network drop in the middle of a transaction.
- **Execution:** tests run in a production-like environment with masked real-time data volumes; concurrency (two users debiting the same account) is tested.
- **Defect management and closure:** each defect has a severity, owner and status; exit criteria (for example, no open critical defects) decide release readiness.

Benefits: (1) Fewer production failures, because high-risk areas are tested first. (2) Consistent, repeatable results across teams and releases. (3) Early defect detection, which reduces the cost of correction. (4) Better traceability and audit compliance, which is essential for banks. (5) Improved customer trust and reliability. Example: a defined process with a rollback test would have caught a defect where money was debited but not credited when the server timed out.

Q3. During development of an e-commerce platform, testing activities are performed inconsistently across phases. This leads to integration and system-level defects. Demonstrate how testing should be carried out as a structured process in SDLC. Explain how process discipline ensures better quality outcomes.

Answer:

Testing should run in parallel with every SDLC phase, as shown by the V-model, rather than being squeezed in at the end.

- **Requirement analysis:** requirements are reviewed for clarity and testability; the acceptance test plan is started (e.g., “customer can apply one coupon per order”).

- **System design:** the system test plan and test scenarios (search, cart, checkout, payment, order tracking) are prepared.
- **High-level design:** integration test cases are designed for module interfaces such as cart to payment gateway and order to inventory.
- **Low-level design / coding:** developers perform unit testing and code reviews; static analysis is used.
- **Testing phase:** unit, integration, system and acceptance testing are carried out in sequence with entry and exit criteria for each level.
- **Deployment and maintenance:** smoke tests, production monitoring and regression testing for every change.

Process discipline and quality outcomes: Clear entry/exit criteria prevent unstable builds from moving to the next level. Each level targets a specific type of defect (unit level: logic errors; integration level: interface errors; system level: end-to-end behaviour), so defects are found where they are cheapest to fix. Documentation and traceability prevent gaps, and metrics make progress visible. The result is fewer integration and system-level defects, predictable releases and higher customer satisfaction.

Q4. A login system shows mismatch between expected and actual outputs during validation. The development team is unsure whether the issue is an error, defect, or failure. Apply basic testing definitions to classify the issue. Explain the relationship between these terms with suitable examples.

Answer:

Definitions:

- **Error (mistake):** a human action that produces an incorrect result, e.g., a developer writes “password != stored” instead of “password == stored”.
- **Defect (bug/fault):** the flaw in the code, design or document that results from the error.
- **Failure:** the observable deviation of the software from its expected behaviour when the defective code is executed.

Classification of the login issue: The mismatch between the expected output (valid user is logged in) and the actual output (valid user is rejected) is observed during execution, so what the tester has found is a **failure**. This failure is caused by a **defect** in the authentication code or

validation logic, and that defect was introduced by a developer's **error** (a wrong condition or misunderstanding of the requirement).

Relationship: Error → Defect → Failure. An error causes a defect; a defect, when executed under particular conditions, may cause a failure. Not every defect causes a failure: a wrong condition in a code path that is never executed stays hidden. Testers observe failures; developers locate and remove the defects; process improvement reduces the errors.

Q5. A team struggles to detect defects early, leading to increased cost during later stages. The manager wants to improve defect detection efficiency. Apply a software testing principle to address this issue. Explain how the principle improves testing effectiveness with an example.

Answer:

Principle applied: Early testing (“shift-left testing”). Testing activities should start as early as possible in the SDLC, beginning with requirements and design, and should be focused on defined objectives.

How it is applied: Testers participate in requirement and design reviews, write test cases while the code is being developed, developers perform unit testing and static analysis, and continuous integration runs automated checks on every build.

How it improves effectiveness: Defects found in the requirement or design stage are cheap to fix, since only a document changes. The same defect found in system testing or in production may require redesign, recoding, retesting and redeployment, so the cost grows many times over at each later stage. Early testing also stops defects from multiplying, since one wrong requirement can create many wrong design elements and code lines.

Example: A requirement states “Orders above a threshold get free delivery” without giving the threshold. A tester finds this ambiguity in the requirement review and gets it clarified (a few minutes of effort). If it had been found after release, the delivery logic would have to be redesigned, fixed, retested and customers compensated.

Q6. During requirement analysis of a healthcare system, several ambiguities go unnoticed. Later, these lead to defects in implementation. Show how a tester can contribute during the requirement phase to improve quality. Explain the importance of early testing involvement.

Answer:

Contribution of a tester in the requirement phase:

- **Requirement review / static testing:** examine the SRS for ambiguity, incompleteness, inconsistency, and untestable statements (e.g., “the system should respond quickly” or “dose should be appropriate”).
- **Raise questions:** ask for exact values, units, ranges and exceptions, for instance the permitted dosage range per age group, or what happens when a patient record is duplicated.
- **Check testability:** every requirement must have measurable acceptance criteria.
- **Prepare early:** derive acceptance test scenarios and a requirements traceability matrix (RTM) from the requirements, which exposes gaps and missing requirements.
- **Risk identification:** highlight high-risk areas such as patient safety, privacy of medical data and regulatory compliance.

Importance of early involvement: Ambiguities removed at this stage prevent wrong designs and wrong code. The cost of correction is lowest, the schedule is protected, testers understand the product well, and the team shares one interpretation of the requirements. In a healthcare system, where a defect may endanger patients, early detection of an unclear requirement is also a safety measure.

Q7. A payroll system produces incorrect outputs due to unclear and ambiguous requirements. The development team needs to identify the origin of defects. Analyze how such defects originate in the software lifecycle. Explain how requirement clarity can prevent such issues.

Answer:

Origin of defects across the lifecycle: Studies show that the largest share of defects originates in the requirement and design phases and not in coding.

- **Requirement phase:** ambiguous statements (e.g., “overtime is paid extra” with no rate or rule), missing rules (tax slabs, leave without pay) and conflicting statements.
- **Design phase:** the designer interprets the unclear requirement in one way and builds the wrong formula or data model.
- **Coding phase:** the developer assumes another interpretation or makes logical mistakes; the wrong rule is hard-coded.
- **Testing phase:** test cases are derived from the same ambiguous document, so the defect is either undetected or reported incorrectly.

The chain is: unclear requirement → human error (misunderstanding) → defect in design/code → failure (wrong salary slip). Because the defect is carried forward, its cost increases at every phase.

How requirement clarity prevents it: Write requirements that are specific, measurable, complete, consistent and testable, with worked examples (e.g., “10 overtime hours at 1.5 × hourly rate = ₹X”). Hold peer reviews and walkthroughs with business users, testers and developers; maintain a traceability matrix; and use prototypes or acceptance criteria to confirm understanding. When all stakeholders interpret a requirement in the same way, the defect never enters the lifecycle.

Q8. A system crashes when boundary input values are entered by users. The team suspects improper testing coverage. Classify this defect and justify your answer. Explain how boundary value analysis could have prevented this issue.

Answer:

Classification: The defect is a **functional (input-validation / boundary condition) defect** of the coding or logic type. Its severity is high because it makes the system crash. The user-visible crash is the failure; the defect behind it is typically an off-by-one error or a wrong relational operator (using < in place of <=), or a missing check for the minimum/maximum value.

Justification: The system works for normal values but fails exactly at or just beside the limits of the valid input range. This pattern, together with the observation that the team suspects poor coverage, indicates that the boundary conditions were never tested or were handled incorrectly in the code. It is a functional defect, as the behaviour deviates from the expected response (a graceful accept/reject message).

How boundary value analysis (BVA) would have prevented it: BVA is a black-box technique based on the fact that defects are most likely at the edges of input partitions. For a field accepting values from 1 to 100, BVA selects 0, 1, 2, 99, 100 and 101. These six test cases check the minimum, maximum and their neighbours, so a crash at 100 or at 0 would have appeared in the first test cycle. Including BVA in the test design would have given systematic coverage of all limits and led to the defect being fixed before release.

Q9. A company maintains a defect repository but rarely uses it for improving testing practices. This results in repeated defects across releases. Demonstrate how a defect repository can support better test case design. Explain its role in improving software quality.

Answer:

A defect repository is a central database that stores every defect with its description, module, severity, root cause, phase of origin, status and fix details. If it is analysed rather than just stored, it becomes a knowledge base for testing.

Support for test case design:

- **Pattern analysis:** identify the modules, features and defect types that fail repeatedly (for example, date handling or null inputs) and add specific test cases for them.
- **New test cases from old defects:** every defect that escaped to production is converted into a test case, which is added to the regression suite so the same defect cannot return.
- **Test prioritisation:** defect-prone, high-severity areas get more test effort and are tested earlier.
- **Checklists and error-guessing:** historical defects supply realistic ideas for negative and boundary scenarios.
- **Root-cause feedback:** if many defects originate from requirements, then early review is strengthened.

Role in quality: It prevents repeated defects across releases, supports metrics such as defect density, defect leakage and defect trend, helps management with release decisions, and promotes continuous process improvement. Example: if three releases had discount-calculation defects at the rounding step, the repository leads to specific rounding test cases being included in all later releases.

Q10. An ATM system fails during withdrawal due to incorrect handling of transaction logic. The issue was not detected during testing. Apply your testing knowledge to identify and describe this as a functional defect. Explain how it could have been prevented.

Answer:

Identification: A functional defect occurs when the software does not behave as specified in the functional requirements. In the ATM, the withdrawal feature is expected to verify the PIN, check the balance and limit, dispense cash and debit the account as a single consistent transaction. If the logic is wrong, for example the account is debited but cash is not dispensed,

a withdrawal exceeding the balance is allowed, or the transaction is not rolled back after a hardware or network error, the system fails to deliver the specified function. This is therefore a functional defect (a logic defect in the transaction-processing module) of high severity, because it directly affects money.

Description: Test input – withdraw ₹5,000 with a balance of ₹4,000. Expected – “Insufficient funds”. Actual – cash dispensed and negative balance. The failure is observed, and the defect lies in the balance-check condition or transaction sequence.

How it could have been prevented:

- Requirement and design review of the transaction rules and exceptional flows.
- Test design using decision tables, state-transition testing (card inserted → PIN verified → amount entered → dispensed → debited) and boundary value analysis for balance and daily limit.
- Negative and fault-injection tests such as power failure, timeout and cash-dispenser jam, to check rollback.
- Code reviews, unit tests on the transaction logic, integration tests with the bank server, and a regression suite after each change.

Q11. A large application shows that most defects are concentrated in a few modules. The testing team wants to optimize their efforts. Use the concept of defect clustering to identify testing priorities. Explain how this improves testing efficiency.

Answer:

Defect clustering is a testing principle which states that a small number of modules usually contain most of the defects (the Pareto principle: about 80% of defects come from 20% of modules).

Identifying testing priorities:

- Collect defect data per module from the defect repository and calculate defect density (defects per KLOC or per function point).
- Rank the modules, and also consider complexity, frequency of change, developer experience and business criticality.
- Classify modules into high, medium and low risk (for example, payment and inventory modules may be high; static help pages low).

- Allocate more testing effort to the high-risk modules: deeper test design, extra reviews, more exploratory and regression tests, earlier testing and static analysis.

Improvement in efficiency: The team finds the maximum number of defects with the minimum effort, uses limited time and resources where they matter most, gets earlier feedback on unstable modules and reduces the risk of critical failures. Developers can also refactor or redesign the problem modules. A caution: the low-risk modules are not ignored entirely, and the clusters should be reviewed regularly, as they may move over time.

Q12. A web application experiences slow performance under heavy user load. The issue needs to be categorized properly. Apply defect classification techniques to identify this type of defect. Explain how classification helps in better resolution.

Answer:

Classification of this defect:

- **By nature/type:** a **non-functional defect – performance (scalability/response-time)** defect. The functions give correct results, but the quality attribute “performance” is not met.
- **By severity:** high (major), as it impacts many users, though the system is not completely down.
- **By priority:** high, because it must be solved before release or at the earliest in production.
- **By origin/phase:** usually a design or implementation defect (inefficient queries, no caching, poor connection pooling, memory leaks, wrong architecture choices) or a missing performance requirement.
- **By detection technique:** found by load and stress testing.

How classification helps in resolution: The defect is routed to the correct owner (database, backend or infrastructure team) and the correct test tool (e.g., JMeter) is used for reproduction and analysis. Severity and priority decide the order of fixing; the type and origin support root-cause analysis and prevention (adding performance requirements, early load testing); and classified data gives metrics that show which kinds of defects occur most frequently. This reduces resolution time and avoids confusion between functional and non-functional problems.

Q13. A team is preparing for regression testing after multiple releases. They have access to historical defect data. Show how this data can be reused effectively for regression testing. Explain how it improves test coverage and efficiency.

Answer:

Regression testing confirms that new changes have not broken existing functionality. Historical defect data helps to build and maintain a focused and effective regression suite.

Ways to reuse the data:

- **Convert fixed defects into regression test cases**, especially those that were found late or in production, so that they are checked in every release.
- **Identify defect-prone modules** (defect clustering) and include more regression tests for them.
- **Impact analysis:** link defects to code components, so that when a component changes, related tests are selected.
- **Prioritise tests** according to severity, frequency and business risk of past defects, and run the critical ones first.
- **Remove redundant or never-failing tests** to keep the suite lean, and select candidates for automation (stable, repetitive, high-value tests).

Improvement in coverage and efficiency: Coverage is increased in the areas where the product has actually failed, and not in randomly chosen areas. Time is saved because the suite is risk-based and not “run everything”. Repeated defects are prevented, release cycles become faster, and teams can show traceability between defects, tests and requirements.

Q14. During integration testing, multiple defects are found due to poor communication between developers and testers. The organization wants to improve collaboration. Identify the role of a tester in minimizing such defects. Explain how early involvement helps.

Answer:

Role of the tester in minimising communication-related defects:

- Take part in requirement, design and interface reviews and clarify assumptions about data formats, APIs and module responsibilities.
- Share test scenarios and test cases with developers early, so that they understand what will be verified.

- Prepare integration test plans and interface checklists (data type, range, sequence, error handling between modules).
- Report defects clearly and reproducibly (steps, expected vs actual result, logs, environment) and keep a polite, fact-based tone.
- Attend daily stand-ups and defect triage meetings, use a common tracking tool and verify fixes quickly.

How early involvement helps: Testers who join from the start know the product, spot gaps and mismatches between modules before coding, and can have the integration environment and test data ready. Developers get quick feedback and understand the tester's viewpoint, so misunderstandings are cleared in the planning stage rather than in integration. This reduces rework, shortens the cycle and builds a "one team for quality" culture.

Q15. A software product repeatedly faces similar defects in every release cycle. The team needs a long-term solution. Apply defect prevention techniques to address recurring issues. Explain how prevention improves overall software quality.

Answer:

Defect prevention focuses on removing the causes of defects, not just finding and fixing them. The following techniques apply to recurring defects:

- **Root cause analysis (RCA)** of every major defect, using the 5-Whys or a fishbone diagram, to find why it happened (unclear requirement, missing check, poor design, lack of knowledge).
- **Defect trend and Pareto analysis** from the defect repository to find the most frequent categories.
- **Corrective and preventive actions:** coding standards and checklists, templates for requirements, design patterns and reusable validated components.
- **Reviews and inspections** of requirements, design and code, and use of static analysis tools.
- **Training and knowledge sharing** on the typical mistakes; retrospectives and lessons-learned sessions after each release.
- **Automated regression and continuous integration** so that a repeated defect is caught immediately.

Improvement in quality: Fewer defects are introduced in the first place, so cost of rework, delays and production incidents fall. Quality becomes predictable from release to release, the team's productivity and morale improve, and customers see a stable product. Prevention is cheaper than detection and correction, and it makes quality a continuous process improvement rather than a last-minute activity.

Q16. A mobile application frequently crashes due to inconsistent testing approaches. The testing team wants to standardize practices. Apply testing principles to redesign the testing strategy. Explain how structured approaches improve reliability.

Answer:

Testing principles applied to the redesigned strategy:

- **Testing shows presence of defects, and exhaustive testing is impossible:** use risk-based selection of features, devices and OS versions, and techniques such as equivalence partitioning and BVA.
- **Early testing:** unit tests and static analysis in development, and early review of the requirements.
- **Defect clustering:** concentrate on crash-prone modules (e.g., camera, location, payments) found through crash analytics.
- **Pesticide paradox:** review and update test cases regularly, add exploratory testing.
- **Testing is context dependent:** mobile-specific tests for interruptions (calls, notifications), low memory, battery, poor or changing network, screen sizes, permissions and app-lifecycle (background/foreground) states.
- **Absence-of-errors fallacy:** also verify usability and performance, not only the absence of crashes.

Strategy elements: a standard test plan and templates; a device/OS matrix chosen from market data; test levels from unit to UAT; automation for regression; beta testing; entry/exit criteria; and crash-log monitoring.

Reliability improvement: Structured and repeatable testing covers the critical scenarios consistently, discovers crash conditions before release, and gives measurable coverage and defect trends, so releases become stable and users experience fewer crashes.

Q17. A company confuses debugging with testing, leading to delays in defect resolution. The team needs clarity on both concepts. Differentiate testing and debugging using a real-time scenario. Explain their roles in improving software quality.

Answer:

Scenario: In an online ticket-booking application, the tester books a ticket and finds that the payment is successful but no confirmation is generated. The tester records the steps and the observed behaviour in the defect tracker. The developer then studies the logs, identifies a null-pointer exception in the confirmation service, corrects the code and hands over the build. The tester re-executes the test and runs regression tests.

Aspect	Testing	Debugging
Purpose	To find defects and check that the software meets the requirements.	To locate the cause of a defect and remove it.
Performed by	Testers (and developers at unit level).	Developers.
Activity	Executing test cases and comparing expected with actual results.	Analysing logs/code, finding the root cause and correcting the code.
Outcome	Defect reports and quality information.	Fixed code and a new build.
Planning	Planned, designed and can be automated.	Usually done on demand; needs code knowledge.
In the scenario	Tester finds that no confirmation is generated after payment.	Developer finds the null-pointer fault and fixes it.

Roles in improving quality: Testing reveals that the software is not behaving as expected and provides information about quality and risk; it is the means of verification and validation. Debugging removes the defect and restores the correct behaviour. Both are needed: testing without debugging only finds problems, and debugging without testing leaves defects hidden and fixes unverified. Together they form the find–fix–verify cycle, which steadily improves product quality.

Q18. An online shopping system behaves differently across browsers. This leads to customer dissatisfaction. Apply testing principles to design a cross-browser testing strategy. Explain how such issues can be effectively handled.

Answer:

Principles applied: exhaustive testing is impossible (so combinations are selected by risk), testing is context dependent (e-commerce relies on UI, payment and responsiveness), early testing, defect clustering (a few browsers/features cause most problems), and the pesticide paradox (the tests are updated regularly).

Cross-browser testing strategy:

- **Define the scope:** pick browsers, versions, operating systems and devices from user analytics (e.g., Chrome, Safari, Firefox, Edge on desktop and mobile).
- **Prioritise:** critical flows first – search, product listing, cart, checkout, payment and login.
- **Test types:** functional behaviour, UI/layout rendering (CSS, fonts, responsive design), JavaScript compatibility, performance, and form/validation behaviour.
- **Tools and environments:** cloud grids such as BrowserStack or Selenium Grid, plus a few real devices; automate repetitive checks.
- **Defect reporting:** record browser, version, OS and screenshot in every defect.

Effective handling: Use web standards and feature detection rather than browser-specific code, follow a progressive-enhancement approach, use CSS resets and polyfills, run cross-browser tests in every build of the CI pipeline, and set a support policy for old browsers. This provides a consistent experience and customer satisfaction.

Q19. A logistics application produces inconsistent outputs due to improper validation. The issue is not detected early. Apply testing concepts to identify gaps in validation. Explain how proper testing improves system reliability.

Answer:

Identifying validation gaps: Compare each input and rule in the requirements with what the application actually enforces, using the following testing concepts:

- **Equivalence partitioning:** test valid and invalid classes for each field (e.g., weight, PIN code, phone number, shipment date, address).

- **Boundary value analysis:** limits such as minimum and maximum weight or package size.
- **Negative testing and error guessing:** blank or null values, wrong data types, special characters, very long strings, negative or zero quantities, past dates, duplicate consignment numbers.
- **Cross-field and business-rule checks:** delivery date must be after dispatch date, destination must be serviceable for the chosen shipping mode, charge must match weight and distance.
- **Interface validation:** data passed between order, warehouse, routing and tracking modules.
- **Requirements traceability matrix** to reveal validation rules which have no test case.

Improvement in reliability: Proper testing makes sure that only correct and complete data enters the system, which means calculations, routing and tracking give consistent outputs. Invalid data is rejected with clear messages, defects are caught early, and failures in production (wrong deliveries, billing errors) are reduced, so the system becomes dependable and trustworthy.

Q20. A team skips structured test case design and directly executes random tests. This leads to poor defect detection. Apply testing as an engineering activity to improve the process. Explain the importance of systematic test design.

Answer:

Applying testing as an engineering activity: The team should follow a disciplined sequence: analyse the requirements and risks → plan the test effort → design test cases using recognised techniques (equivalence partitioning, BVA, decision tables, state-transition, use-case testing) → set up the test data and environment → execute and log results → report and track defects → measure coverage and improve. Reviews of test cases and entry/exit criteria ensure the quality of the testing itself.

Importance of systematic test design:

- **Better defect detection:** test cases are deliberately chosen to expose defects at boundaries, in invalid classes and in rule combinations.
- **Coverage:** requirements and code coverage can be measured, so gaps are visible and nothing is forgotten.

- **Repeatability and consistency:** any tester can run the same tests and obtain comparable results; useful for regression.
- **Efficiency:** a small, well-chosen set of tests replaces large numbers of redundant random tests, saving time and cost.
- **Traceability and accountability:** each test is linked to a requirement and provides evidence for the audit and for release decisions.
- **Reuse:** documented test cases can be reused and improved in later releases.

Example: for a discount rule “10% for orders of ₹1,000 or more”, systematic design tests ₹999, ₹1,000 and ₹1,001, whereas random testing may never try these values.

Q21. A healthcare system shows data inconsistency between modules due to integration issues. The testing team needs to analyze the problem. Apply testing principles to identify root causes. Explain how proper testing ensures data consistency.

Answer:

Applying testing principles to find root causes:

- **Testing shows presence of defects:** reproduce the inconsistency with a specific case (e.g., a patient’s allergy updated in the registration module but not shown in the pharmacy module).
- **Defect clustering:** check the interfaces which generate most issues (patient registration, lab, billing, pharmacy).
- **Root-cause analysis:** trace the data flow module by module and examine logs and database records to find where values change or are lost.
- **Typical causes:** mismatched data types or formats between modules, wrong field mapping, different units or code sets, missing or partial updates (no transaction control), concurrent updates, duplicate or stale records, caching problems and unclear interface specifications.

How proper testing ensures consistency:

- **Integration testing** (top-down, bottom-up or sandwich) with interface and data-flow tests on every module pair.
- **End-to-end scenario tests** that follow one patient record through all modules.
- **Database and reconciliation checks** to compare the records of the source and the target systems; tests for transaction rollback (ACID properties) and concurrent access.

- **Controlled, realistic test data**, and regression tests after every interface change.

This detects mismatches before release, keeps patient data accurate and reliable across modules, and protects patient safety.

Q22. A project experiences delays because testers are not involved early in development. The organization wants to improve collaboration. Explain the role of testers across SDLC phases. Show how early involvement improves quality.

Answer:

Role of testers across SDLC phases:

- **Requirement analysis:** review requirements for ambiguity and testability, identify risks, define acceptance criteria and start the test strategy.
- **Design:** review the design and interfaces, prepare the system and integration test plans and scenarios.
- **Coding/implementation:** design detailed test cases and test data, set up the test environment, support unit testing and code/static reviews, prepare automation scripts.
- **Testing:** execute integration, system and acceptance tests, log and verify defects, perform regression and report progress and quality metrics.
- **Deployment:** carry out smoke/sanity tests, support UAT and release readiness decisions.
- **Maintenance:** regression testing for fixes and enhancements, analysis of production defects.

How early involvement improves quality: Defects are found when they are cheapest to fix; testers know the product and are ready to test as soon as the build arrives; the test cases are ready early, so the testing phase is shorter and delays are avoided; communication gaps with developers and analysts are removed; and risk-based planning improves coverage. The overall result is a higher-quality product delivered on schedule with less rework.

Q23. A system fails during peak usage due to untested scenarios. The testing team missed critical cases. Apply testing principles to identify gaps. Explain how better test coverage prevents such failures.

Answer:

Identifying gaps using testing principles:

- **Exhaustive testing is impossible, so risk-based testing is needed:** peak-load scenarios (sales events, month-end processing, exam result days) are high risk and should have been identified and prioritised.
- **Testing is context dependent:** a high-traffic system needs non-functional tests (load, stress, spike, endurance, scalability), not only functional tests.
- **Early testing and traceability:** compare requirements, expected user volumes and the operational profile with the existing test cases; the requirement-to-test matrix will show uncovered scenarios, such as concurrent logins, simultaneous transactions, resource contention, session timeouts and database locking.
- **Defect clustering and production data:** examine logs and past incidents to find what failed before.

How better coverage prevents failures: Requirement, scenario, risk and non-functional coverage ensures that realistic peak conditions are exercised before release. Load and stress tests reveal bottlenecks, memory leaks and thread/DB limits; capacity can be adjusted in advance; and failure behaviour (graceful degradation, recovery) can be verified. Coverage metrics show how much has been tested, so the team can add the missing cases and release with confidence.

Q24. A software team lacks proper defect documentation, making tracking difficult. The organization wants to improve defect management. Apply defect repository concepts to solve this issue. Explain how documentation enhances tracking and analysis.

Answer:

Applying defect repository concepts: Introduce a central defect-tracking tool (such as Jira or Bugzilla) and make it compulsory to record every defect with standard fields:

- Unique ID, title/summary and detailed description.
- Steps to reproduce, expected result and actual result, with screenshots or logs.
- Environment (build, OS, browser/device) and module/feature.
- Severity, priority, type/category and phase in which it was introduced and detected.
- Reporter, assignee, dates and status (New → Assigned → Open → Fixed → Retest → Verified → Closed/Reopened/Rejected/Deferred).
- Root cause and fix details.

How documentation enhances tracking and analysis: Each defect can be followed through its life cycle, with clear ownership and no loss of information. Developers can reproduce and fix faster. Management can see open defects, ageing and release readiness. The stored data supports metrics such as defect density, defect leakage, defect removal efficiency and trends, and root-cause and clustering analysis. It also supplies lessons for regression suites and prevention, and gives audit evidence.

Q25. A banking system produces repeated transaction failures due to improper testing practices. The team needs to improve quality. Apply software testing principles to redesign the testing approach. Explain how structured testing enhances reliability.

Answer:

Redesigned approach by applying testing principles:

- **Early testing:** review the transaction requirements (limits, charges, cut-off times) and begin test design in the requirements phase.
- **Risk-based testing:** fund transfer, withdrawal, deposit, interest and settlement are the highest-risk functions and receive the most effort.
- **Systematic design:** boundary value analysis for amounts and limits, decision tables for rules, and state-transition testing for the transaction life cycle (initiated, authorised, posted, reversed).
- **Negative and failure tests:** network failure, timeout, duplicate submission, insufficient balance, and rollback or reversal.
- **Non-functional tests:** concurrency, performance and security (authentication, authorisation, data protection).
- **Regression and pesticide paradox:** an automated, regularly updated regression suite; use of production-like environment and data volumes.
- **Process control:** defined entry/exit criteria, traceability, defect tracking and root-cause analysis for any production failure.

Reliability enhancement: The structured approach covers critical paths and exceptional cases in a repeatable way, so failures are discovered before release, repeated defects are prevented, accounts stay consistent and customer confidence and compliance are strengthened.

Q26. A system shows failures due to missing edge case testing. The testers focused only on normal inputs. Apply testing principles to include edge cases. Explain how boundary testing improves defect detection.

Answer:

Applying testing principles: Since exhaustive testing is impossible, testers must select the inputs that are most likely to reveal defects. Equivalence partitioning divides the inputs into valid and invalid classes, and defect clustering at the edges of these classes is the reason to include **edge cases**: minimum and maximum values, just inside and outside the limits, empty or null input, zero, negative values, maximum field length, special characters, first and last items in a list, leap-year and month-end dates and very large files or quantities. Both positive and negative tests are needed, as testing only “normal” inputs gives a false sense of security.

Boundary value testing: For a field accepting an age from 18 to 60, the test values are 17, 18, 19, 59, 60 and 61 (two-value or three-value BVA).

How it improves defect detection: Many defects are caused by off-by-one errors, a wrong relational operator (< instead of <=), wrong loop limits or missing range checks, and these appear only at the boundaries. A small number of boundary tests therefore finds a large share of defects at low cost, gives good coverage of the input domain with few test cases, and prevents crashes and wrong outputs at the limits when the system is in use.

Q27. A development team struggles to maintain consistency in testing across modules. This leads to uneven quality. Apply process-based testing approaches to address this issue. Explain how standardization improves outcomes.

Answer:

Process-based testing approaches:

- **Define a common test process and policy** for all modules: planning, analysis and design, execution, defect reporting and closure.
- **Standard templates and guidelines:** test plan, test case, defect report and test summary formats; naming conventions and a test case design technique guideline.
- **Entry and exit criteria** for every test level (unit, integration, system, acceptance) and a common definition of “done”.
- **Reviews and inspections** of test cases across modules, and a traceability matrix for requirement coverage.

- **Central test management and defect tools**, and shared test data and environments.
- **Metrics and monitoring:** coverage, defect density, pass rate and leakage compared module by module.
- **Process improvement and training** based on recognised models/standards (such as TMMi or ISO/IEC/IEEE 29119) and lessons learned.

How standardization improves outcomes: All modules are tested to the same depth, so quality is uniform and there are no weak areas. Results are comparable and predictable, new team members become productive quickly, reuse of tests and tools increases, duplication and rework are reduced, and management has accurate data for decision-making. Consistency also reduces dependence on individuals and raises the overall product quality.

Q28. A web application fails due to improper handling of unexpected inputs. The testing team missed validation scenarios. Apply testing concepts to improve input validation. Explain how proper testing ensures robustness.

Answer:

Applying testing concepts to improve input validation:

- **Equivalence partitioning and boundary value analysis** for each input field (length, range, format).
- **Negative testing and error guessing:** blank or null values, wrong data types (text in a numeric field), special characters, extremely long strings, negative numbers, invalid email/phone/date formats, wrong file types and sizes.
- **Security-oriented input tests:** SQL injection, cross-site scripting (XSS) strings, script tags and unusual encodings; random/fuzz inputs.
- **Both client-side and server-side validation:** bypass the browser checks and send invalid values directly to the server.
- **Error-message and recovery checks:** the application must show clear messages, retain valid data and never crash or expose system details.
- **Requirement review** to list all validation rules, and a traceability matrix to make sure that each has tests.

How proper testing ensures robustness: Robustness is the ability to continue working correctly under invalid or unexpected conditions. Exercising the system with such inputs shows where it crashes, hangs, corrupts data or opens security holes, so these are fixed before release.

The application then handles bad input gracefully, the data remains clean and secure, and users have a stable experience.

Q29. A software company faces frequent defect leakage into production. The management wants to reduce such issues. Apply testing principles to analyze the root cause. Explain how improved testing practices prevent leakage.

Answer:

Defect leakage means defects which escape the testing phases and are found by users in production. It is measured as: $\text{Defect Leakage (\%)} = (\text{defects found in production} \div \text{total defects found before and after release}) \times 100$.

Root-cause analysis using testing principles:

- **Early testing:** were the requirements reviewed, or did ambiguous requirements cause the defects?
- **Exhaustive testing is impossible / risk-based testing:** were the high-risk areas and critical scenarios covered?
- **Defect clustering:** do the leaked defects belong to a few modules?
- **Pesticide paradox:** are the same old test cases being repeated without updates?
- **Context dependency:** were non-functional, compatibility and production-like environment and data tested?
- Other typical causes are inadequate regression after late changes, schedule pressure, poor traceability, weak UAT and untested configurations.

Each leaked defect should be analysed by 5-Whys or a fishbone diagram to find out why it was not detected earlier.

Prevention through improved practices: reviews and static testing, a requirements traceability matrix, risk-based and technique-based test design, new test cases for every escaped defect, an automated regression suite, a production-like test environment, strict exit criteria, UAT and beta testing, and monitoring of leakage metrics. These measures detect defects before release and steadily reduce leakage.

Q30. A project team is unable to identify defects early due to poor testing strategies. This increases overall cost and effort. Apply testing principles to redesign the strategy. Explain how early defect detection improves efficiency.

Answer:

Redesigned strategy using testing principles:

- **Early testing (shift-left):** involve testers in the requirement and design stages; use reviews, walkthroughs and inspections (static testing) for all work products.
- **Test planning with risk analysis:** prioritise the high-risk and business-critical features, as exhaustive testing is not possible.
- **Early test design:** derive test cases from the requirements as they are written, using standard techniques and a traceability matrix.
- **Strong unit and integration testing:** developers write unit tests, and continuous integration with automated builds and tests gives fast feedback.
- **Defect clustering and pesticide paradox:** use defect data to focus on weak areas and refresh the tests regularly.
- **Defined metrics and exit criteria:** defect detection percentage, coverage and defect trend by phase.

How early detection improves efficiency: The cost of fixing a defect increases sharply with each phase (a defect found at requirement stage may cost 1 unit, at coding 5–10, in system testing 20–50 and in production 100 or more). Early detection avoids the spread of the defect into design and code, reduces rework, shortens the schedule, lowers the number of production incidents and frees the team to build new functionality. The project therefore finishes with lower cost and effort and higher quality.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand